

Apellidos, Nombre:Lucas Lillo
Apellidos, Nombre:Daniel Flores
Apellidos, Nombre:

La entrega definitiva de este proyecto será el 11 de mayo. En la entrega, subid:

- **Este fichero (doc/pdf) con las respuestas/código e imágenes pedidas.**
- **El fichero GIF correspondiente al morphing con vuestras propias imágenes (apartado 2.4)**

Para la clase del 4 de mayo debéis llevar hecho hasta el punto indicado. En esa clase revisaré/puntuaré lo que tengáis hecho, resolveremos dudas, y seguiréis trabajando con el resto de los apartados, ...

Proyecto de Transformaciones Geométricas

1) Deformaciones lineales

Obtención de la matriz P de una transformación proyectiva

Usad la función `show_mat(A)` para volcar la transformación iP resultante (que pasa de coordenadas u,v a x,y).

Matriz:

```
1.2529  0.4858 -359.13
-0.5000  1.0000  107.50
0.0000  0.0000   1.00
```

Volcad la matriz P obtenida con `show_mat`.

Matriz:

```
0.1865  1.3260 -91.08
-1.3170  2.1364 298.09
-0.0014  0.0021   1.00
```

¿De qué orden son ahora las diferencias entre las coordenadas u,v originales y las obtenidas al aplicar P a (x,y) ?

Diferencias (du,dv) :

$1.0e-12 *$

```
-0.0422    0
-0.1137 -0.0995
-0.0568    0
  0        0
```

Como indican los vectores de las diferencias entre u y v antes y después de aplicar P, la diferencia es de valores inferiores a 10^{-13} unidades.

Adjuntad código de vuestra función `get_proy` una vez verificada.

`function P=get_proy(x,y,u,v)`

```

x=x'; y=y'; u=u'; v=v';
z=0*x; unos=x.^0; % 0's y 1's

M = [x y unos z z z -u.*x -u.*y;
     z z z x y unos -v.*x -v.*y];

uv=[u;v];
sol= M\uv;

P = [sol(1:3)';
     sol(4:6)';
     sol(7:8)' 1];

end

```

¿A qué coordenadas u,v iría a parar un punto con coordenadas $(x=200,y=100)$ al aplicarle la transformación proyectiva dada por P ?

Punto (200,100) transformado:

84.6977 266.8205

Volcad ahora la matriz obtenida con `get_proy(u,v,x,y)`, que pasa de coordenadas destino (u,v) a origen (x,y) . Volcad también la matriz inversa (inv) de la matriz P anterior $(x,y \rightarrow u,v)$. ¿Son iguales?

`P2 = get_proy(u,v,x,y)`

Matriz:

```

0.6975 -0.7094 275.00
0.4166 0.0266 30.00
0.0001 -0.0011 1.00

```

Inv de P

Matriz:

```

1.0352 -1.0530 408.17
0.6183 0.0395 44.53
0.0001 -0.0016 1.48

```

Proporcion P2 y inv P

```

0.6737 0.6737 0.6737
0.6737 0.6737 0.6737
0.6737 0.6737 0.6737

```

No son iguales pero como se puede apreciar con la comprobación $(P2 ./ \text{Inv de } P)$ son proporcionales

Deformación de una imagen usando transformaciones lineales

[Adjuntad código de warp_img.](#)

```
function im2 = warp_img(im, iP)

% Dimensiones de la imagen
[N, M, C] = size(im);

% Reservamos la imagen de salida y matrices de coordenadas a interpolar
im2 = zeros(N, M, C);
X = zeros(N, M);
Y = zeros(N, M);

% Recorremos las coordenadas en la imagen de salida
for u = 1:M
    for v = 1:N
        % Aplicamos transformación inversa
        [x, y] = convertir(u, v, iP);
        X(v, u) = x;
        Y(v, u) = y;
    end
end

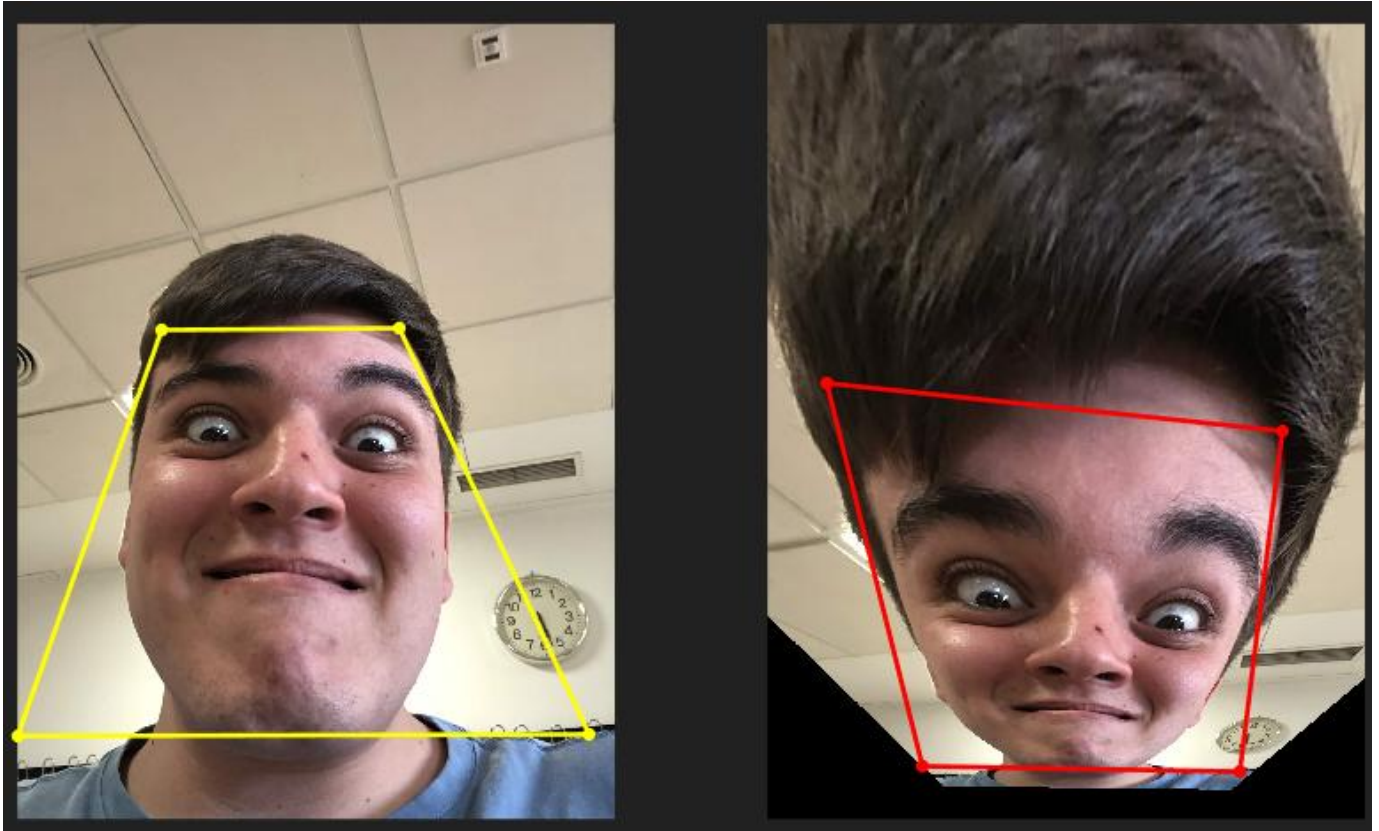
% Interpolamos cada plano de color por separado
for c = 1:C
    im2(:, :, c) = interp2(im(:, :, c), X, Y, 'bicubic');
end

end % end de la función
```

[Adjuntad la imagen resultante.](#)



Adjuntad captura de demoP4 donde se vea la imagen original y la deformada. Volcad las coordenadas de los vértices elegidos y calculad la matriz P que define la deformación mostrada.



Adjuntad captura del resultado.

Vértices derecha : (522 1107) (1381 1101) (2067 2569) (1 2575)

Vértices izquierda: (214 1300) (1862 1473) (1708 2704) (561 2684)

----- Llevad hecho hasta aquí para la próxima clase -----

Imágenes recursivas

Volcad (show_mat) la matriz P de dicha transformación.

Matriz P del cartel:

Matriz:

```
0.0930 -0.0171 586.40
-0.1084 0.3371 270.74
-0.0003 -0.0000 1.00
```

Adjuntad la imagen resultante.

La imagen resultante que obtenemos es:

Transformación dentro del espacio del cartel



Adjuntad código usado para fusionar ambas imágenes y captura de la imagen final resultante.

El código que hemos usado para la fusión, sin usar bucles es:

```
% == Fusión de las imágenes ==
% Calcula la mascara de la imagen
mascara = ~isnan(im_deformada);
% Borrarnos la zona del cartel en la imagen original
im_fusion = im_cartel;
im_fusion(mascara) = 0;
% Pegamos la imagen deformada en la zona del cartel
im_fusion(mascara) = im_deformada(mascara);
figure; imshow(im_fusion); title('Imagen fusionada');
```

PERO, está basado en el código del primer apartado para obtener la transformación de la imagen. Que es el siguiente:

```
% == Transformación de la imagen dentro del espacio del cartel ==
% Cargamos billboard.jpg
im_cartel = double(imread('billboard.jpg')) / 255;
[N, M, ~] = size(im_cartel);

% Medimos las esquinas del cartel
figure; imshow(im_cartel);
title('Haz clic en las 4 esquinas que queramos: Superior-Izq, Superior-Der, Inferior-Der, Inferior-Izq');
hold on;

u_cartel = zeros(1,4);
```



```

v_cartel = zeros(1,4);
for k = 1:4
    [u_cartel(k), v_cartel(k)] = ginput(1);
    plot(u_cartel(k), v_cartel(k), 'g.', 'MarkerSize', 20);
end
hold off;

% Ajustamos las coordenadas
x_cartel = [1    M    M    1    ];
y_cartel = [1    1    N    N    ];
% Calculamos P
P = get_proy(x_cartel, y_cartel, u_cartel, v_cartel);
fprintf('Matriz P del cartel:\n');
show_mat(P);

iP = inv(P);
im_deformada = warp_img(im_cartel, iP);
figure; imshow(im_deformada); title('Transformación dentro del espacio del cartel');

```

Imagen obtenida:



¿Se os ocurre cómo repetir el proceso para insertar copias adicionales de la foto dentro del nuevo cartel creado SIN TENER QUE VOLVER A MEDIR más coordenadas? La solución para esto sería aplicar la misma P una y otra vez sobre la imagen que ya hemos fusionado. Esta vez, si hemos usado bucles:

```

% Repetimos el proceso sin volver a medir la zona del cartel
im_bucle = im_fusion;
for copias = 1:3 % Numero de veces que queremos repetir el proceso
    im_deformada_k = warp_img(im_bucle, iP);

```

```
mascara_k = ~isnan(im_deformada_k(:,:,1));  
for c = 1:size(im_bucle, 3)  
    canal = im_bucle(:,:,c);  
    canal_d = im_deformada_k(:,:,c);  
    canal(mascara_k) = canal_d(mascara_k);  
    im_bucle(:,:,c) = canal;  
end  
end  
figure; imshow(im_bucle); title('Imagen recursiva');
```

Resultado:



Si hacemos zoom se llegamos a apreciar el cartel original:



2. Aplicación: Morphing

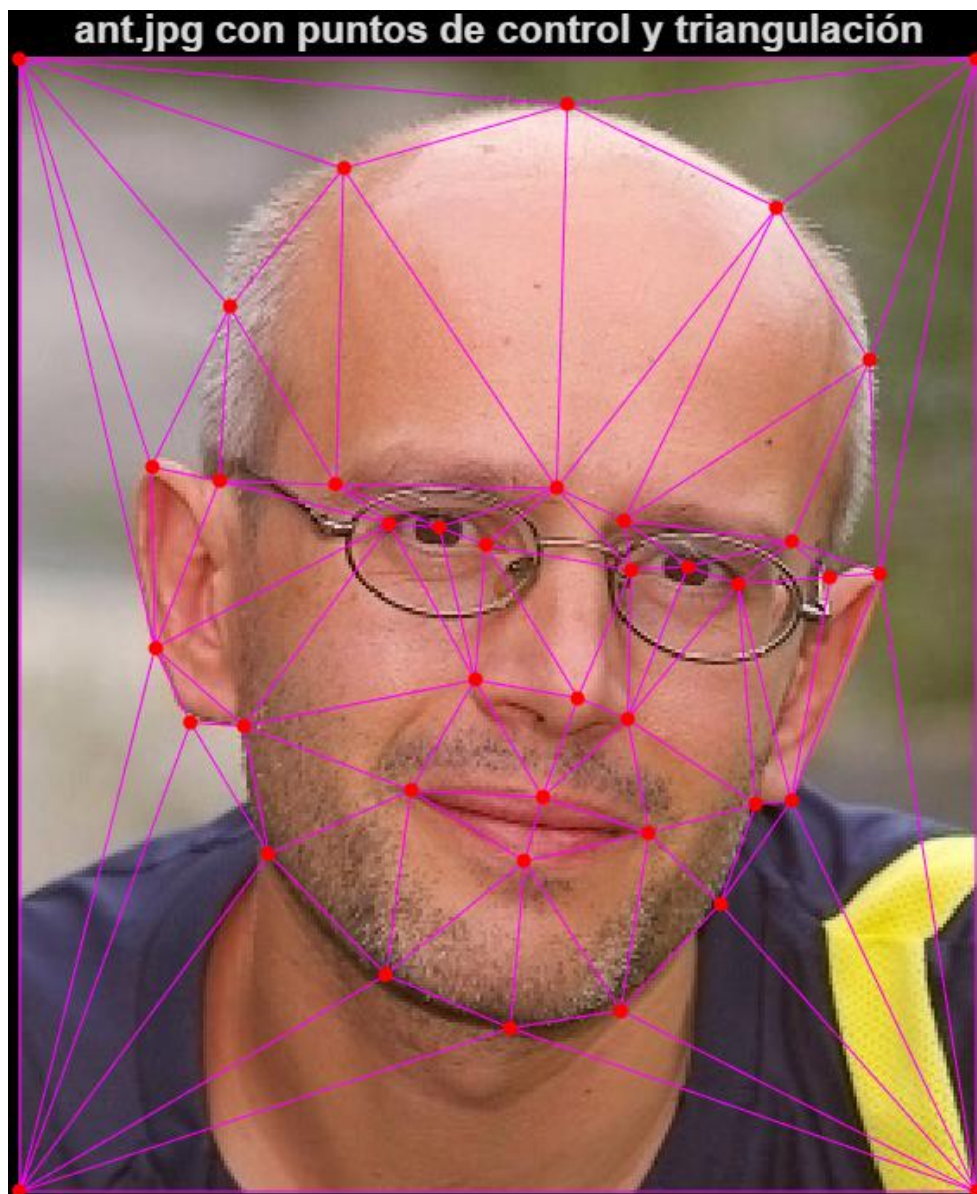
a) Triangulación y cálculo de las matrices de transformación

¿Cuántos triángulos tiene la triangulación obtenida?

En la triangulación obtenida obtenemos un total de 74 triángulos

Número de triángulos NT = 74

Adjuntad una captura de la segunda foto mostrando sus puntos de control y la triangulación.



Adjuntad código del bucle para calcular las matrices de transformación iP1 e iP2.

```
% Arrays de celdas para las NT matrices inversas  
iP1 = cell(1, NT);
```

```

iP2 = cell(1, NT);

for k = 1:NT
    % 1) Índices de los vértices del triángulo k
    idx = T(k, :);
    % 2) Coordenadas de origen en im1 e im2, osea los vertices del triangulo
    X1 = x1(idx); Y1 = y1(idx); % vértices en imagen 1 (willis)
    X2 = x2(idx); Y2 = y2(idx); % vértices en imagen 2 (ant)
    % destino
    U = u(idx); V = v(idx);

    iP1{k} = get_afin(U, V, X1, Y1);
    iP2{k} = get_afin(U, V, X2, Y2);
end

% mosttramos las matrices del triángulo nº 3
fprintf('\niP1 triángulo 3:\n'); show_mat(iP1{3});
fprintf('\niP2 triángulo 3:\n'); show_mat(iP2{3});

```

Volcad en la hoja de respuestas (usando show_mat) las matrices obtenidas para iP1 e iP2 correspondientes al triángulo nº 3 de la triangulación.

iP1 triángulo 3:

Matriz:

```

0.8141 0.0045 11.81
-0.1275 0.9481 17.36
0.0000 0.0000 1.00

```

iP2 triángulo 3:

Matriz:

```

1.1859 -0.0045 -11.81
0.1275 1.0519 -17.36
0.0000 0.0000 1.00

```

b) Escribir una rutina para deformar una imagen

Adjuntad las líneas de código modificadas.

Linea de la versión anterior

```
[x, y] = convertir(u, v, iP); % iP es una sola matriz
```

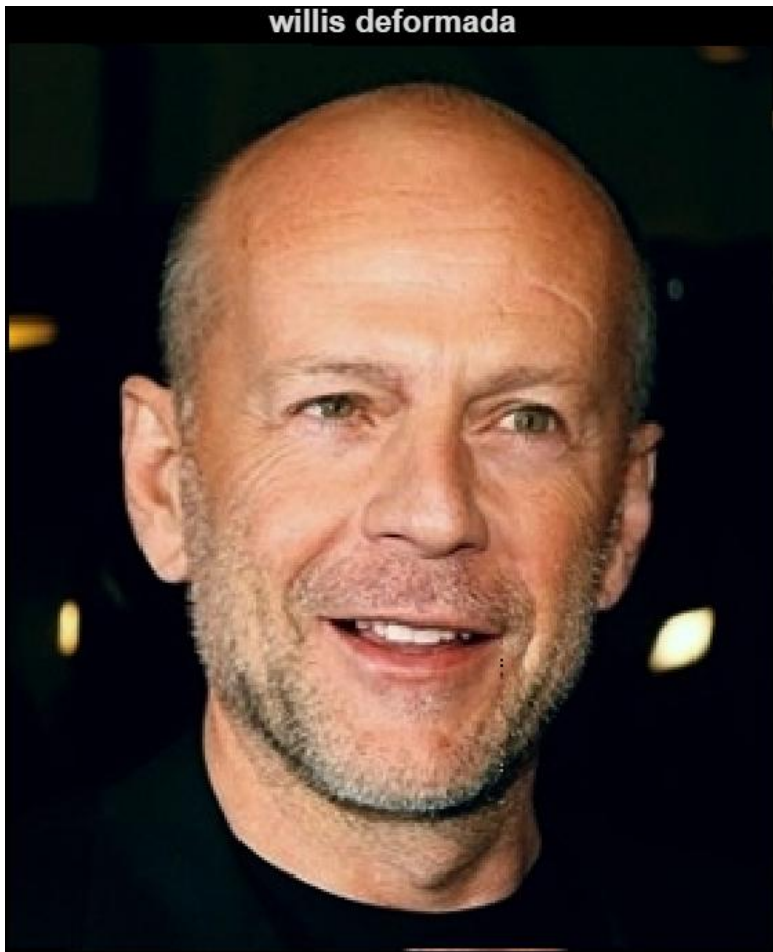
Linea de la nueva versión

```

idx = zona(v, u); % línea añadida: índice del triángulo
[x, y] = convertir(u, v, iP{idx}); % iP{idx} selecciona la matriz correcta

```

Adjuntar las dos imágenes deformadas.



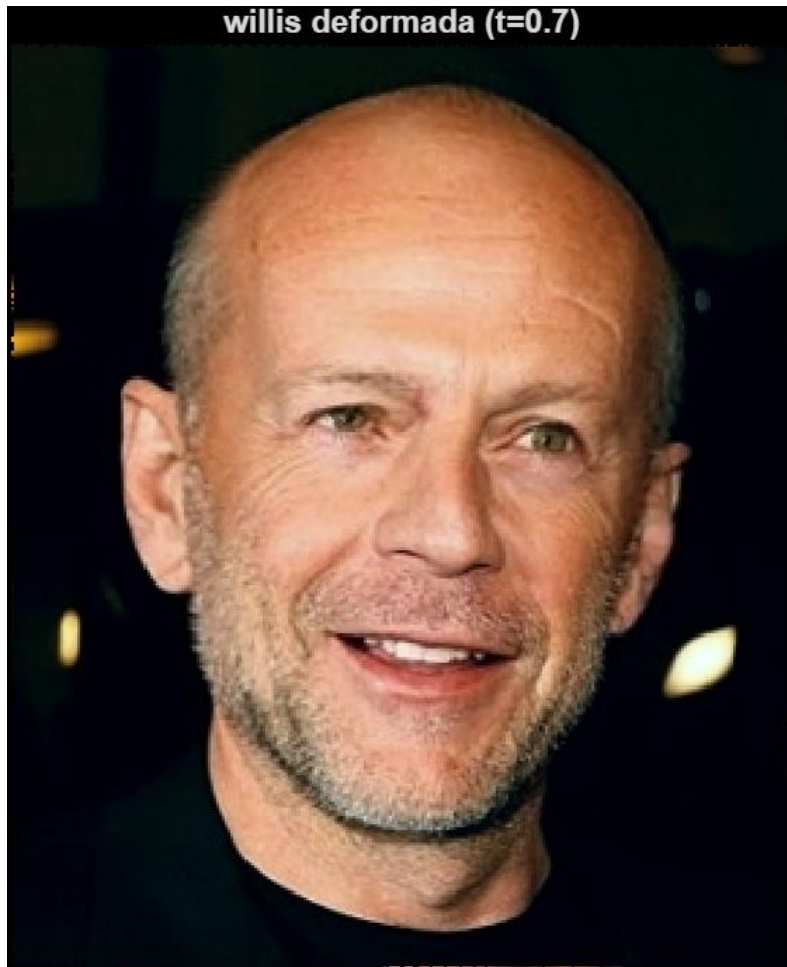


Adjuntad una captura del resultado.



Adjuntad las imágenes deformadas y la mezcla obtenida para $t=0.7$, junto con todo el código usado para crearla (sin las funciones ya incluidas).





El código utilizado es el siguiente:

```
% Cambiamos el valor de t a 0.7
t = 0.7;
% Recalculamos los puntos destino (u,v)
u_t = (1-t) * x1 + t * x2;
v_t = (1-t) * y1 + t * y2;

% Recalculamos las matrices iP1 e iP2 con los nuevos puntos
iP1_t = cell(1, NT);
iP2_t = cell(1, NT);
for k = 1:NT
    idx = T(k, :);
    X1 = x1(idx); Y1 = y1(idx);
    X2 = x2(idx); Y2 = y2(idx);
    U = u_t(idx); V = v_t(idx);
    iP1_t{k} = get_afin(U, V, X1, Y1);
    iP2_t{k} = get_afin(U, V, X2, Y2);
end
% Recalculamos zona con los nuevos (u_t, v_t)
zona_t = determina_triang(T, u_t, v_t, N, M);
% Deformamos las imagenes
im1_d = warp_img_trozos(im1, iP1_t, zona_t);
im2_d = warp_img_trozos(im2, iP2_t, zona_t);
figure; imshow(im1_d); title('willis deformada (t=0.7)');
figure; imshow(im2_d); title('ant deformada (t=0.7)');
% Mezcla ponderada final
res = (1-t) * im1_d + t * im2_d;
figure; imshow(res); title('Mezcla final morphing t=0.7');
```

c) Optimización de la rutina warp_img_trozos()

Adjuntad código de vuestra función optimizada.

```
function im2 = warp_img_trozos_opt(im, iP, zona)

% Dimensiones de la imagen
[N, M, C] = size(im);

% Reservamos las matrices de coordenadas a interpolar
X = zeros(N, M);
Y = zeros(N, M);

NT = length(iP);

% Recorremos los triángulos de la imagen de salida
for k = 1:NT
    % Consultamos qué píxeles pertenecen al triángulo
    pt = find(zona == k);
```

```
if isempty(pt), continue; end

% Convertimos índices lineales a coordenadas de imagen
[v, u] = ind2sub(size(zona), pt');

% Aplicamos transformación inversa
[x, y] = convertir(u, v, iP{k});

X(pt) = x;
Y(pt) = y;
end

% Interpolamos cada plano de color por separado
im2 = zeros(N, M, C);
for c = 1:C
    im2(:, :, c) = interp2(im(:, :, c), X, Y, 'bicubic');
end

end
```

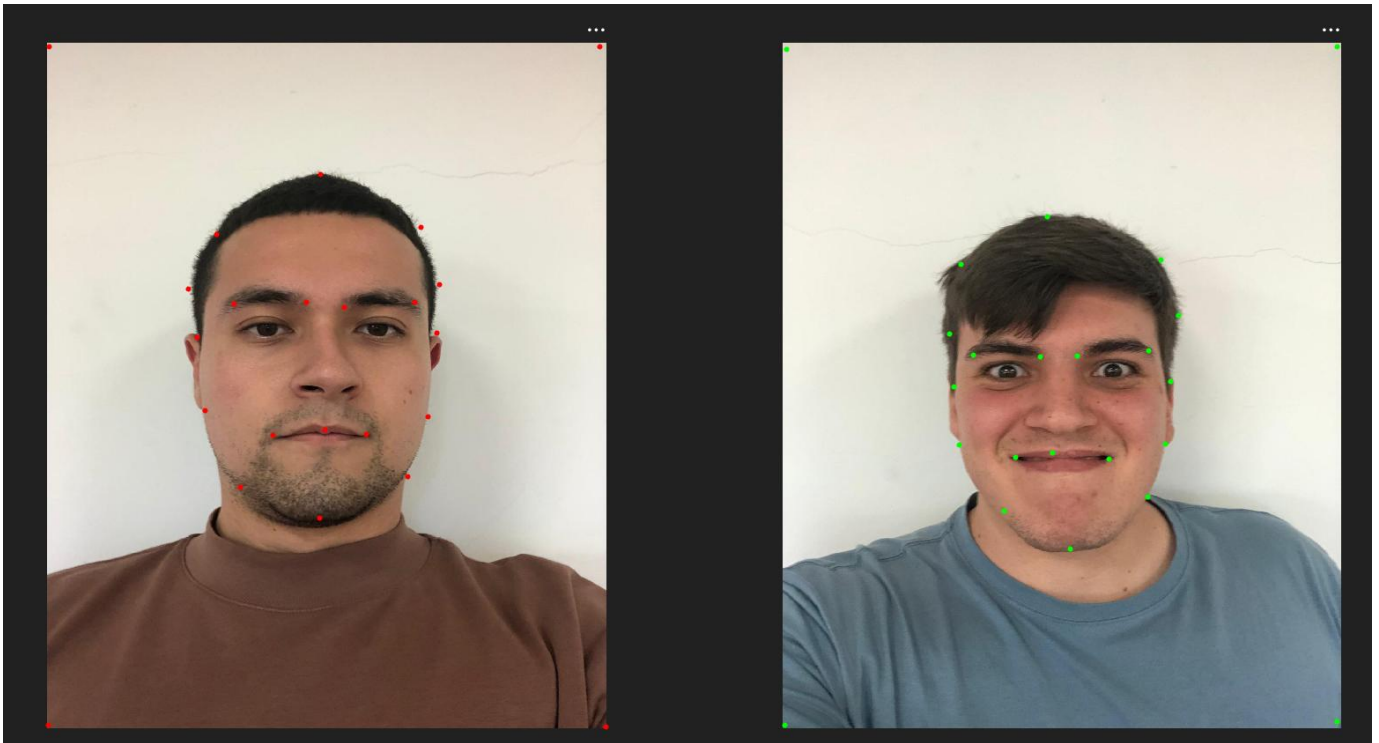
Usando tic/toc medir su tiempo de ejecución para deformar 100 veces una imagen y compararlo con el de la versión no optimizada. Volcad los tiempos obtenidos.

Los resultados que obtenemos son:

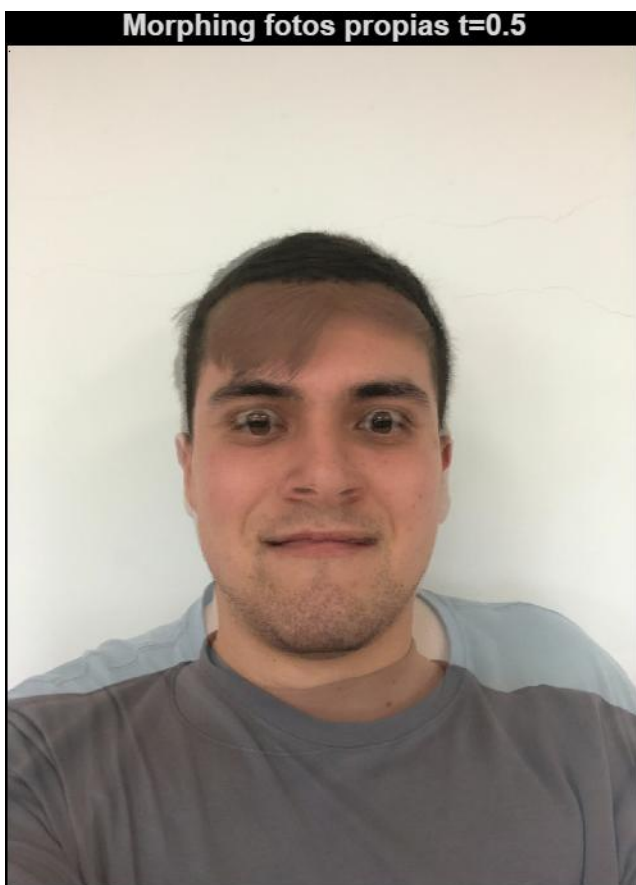
Versión no optimizada (100x): 3.740 s → 0.0374 s/iter
Versión optimizada (100x): 1.953 s → 0.0195 s/iter
Aceleración: x1.9

d) Morphing con vuestras propias imágenes

Adjuntad las imágenes originales con los puntos de control que habéis marcado superpuestos sobre ellas.



Adjuntad la imagen mezcla (imágenes si sois más de 2) para el punto medio ($t=0.5$) entre cada pareja de fotos originales.



Subid el GIF resultante en la entrega.

En Word no se llega a ver la animación pero se puede descargar el GIF desde nuestra entrega de Moodle.



3) Transformaciones no lineales y sus inconvenientes

Adjuntad el código de la función.

```
function P = get_nolineal(x,y,u,v)
```

```
x = x';
y = y';
u = u';
v = v';
```

```
z = 0*x;
unos = x.^0;
```

```
xy = x .* y;
```

```
M = [unos x y xy z z z z;
      z z z z unos x y xy];
```

```
uv = [u;v];
```

```
sol = M \ uv;
```

```
P = [sol(1:4)';
     sol(5:8)'];
```

```
end
```

Volcar la matriz P con los coeficientes para la transformación directa $(x,y \rightarrow u,v)$.

Matriz:

```
-1175.14  4.3319  1.6846 -0.00482
-398.24  1.1589  1.4378 -0.00142
```

Adjuntad código de vuestra función convertir() modificada.

```
function [u,v] = convertir(x,y,P)
```

```
x = x(:)';
y = y(:)';
```

```
% CASO LINEAL
```

```
if numel(P) == 9
```

```
    X = [x;
         y;
         ones(1,length(x))];
```

```
    U = P * X;
```

```
    u = U(1,:) ./ U(3,:);
    v = U(2,:) ./ U(3,:);
```

```
% CASO NO LINEAL
```

```
else
```

```
    X = [ones(1,length(x));
         x;
         y;
         x.*y];
```

```
    U = P * X;
```

```
    u = U(1,:);
    v = U(2,:);
```

```
end
```

```
end
```

Adjuntad los vectores con las diferencias entre las coordenadas u,v originales y las U,V obtenidas al aplicar convertir() a las coordenadas origen (x,y) .

Diferencias entre (u,v) y (u2,v2):

1.0e-12 *

```

0.0568    0
0.1137    0.1137
    0    0.3411
-0.3411   -0.1137

```

Son prácticamente iguales como se esperaba, probablemente debido a problemas de calculo a la hora de realizar varias transformaciones por parte de matlab

Dad la matriz iP (2x4) obtenida de esta forma.

Matriz:

```

510.43 -0.6087 -1.1520  0.00355
119.92 -0.3924  0.9157  0.00046

```

Adjuntad vector con las diferencias entre las coordenadas X,Y recuperadas y las x,y originales.

Diferencias entre (x,y) y (x2,y2):

1.0e-13 *

```

    0    0
0.5684    0
    0    0
0.5684    0

```

Adjuntad una captura del resultado.



Volcad las coordenadas (X,Y) a las que "regresamos" (que ya no son las mismas).

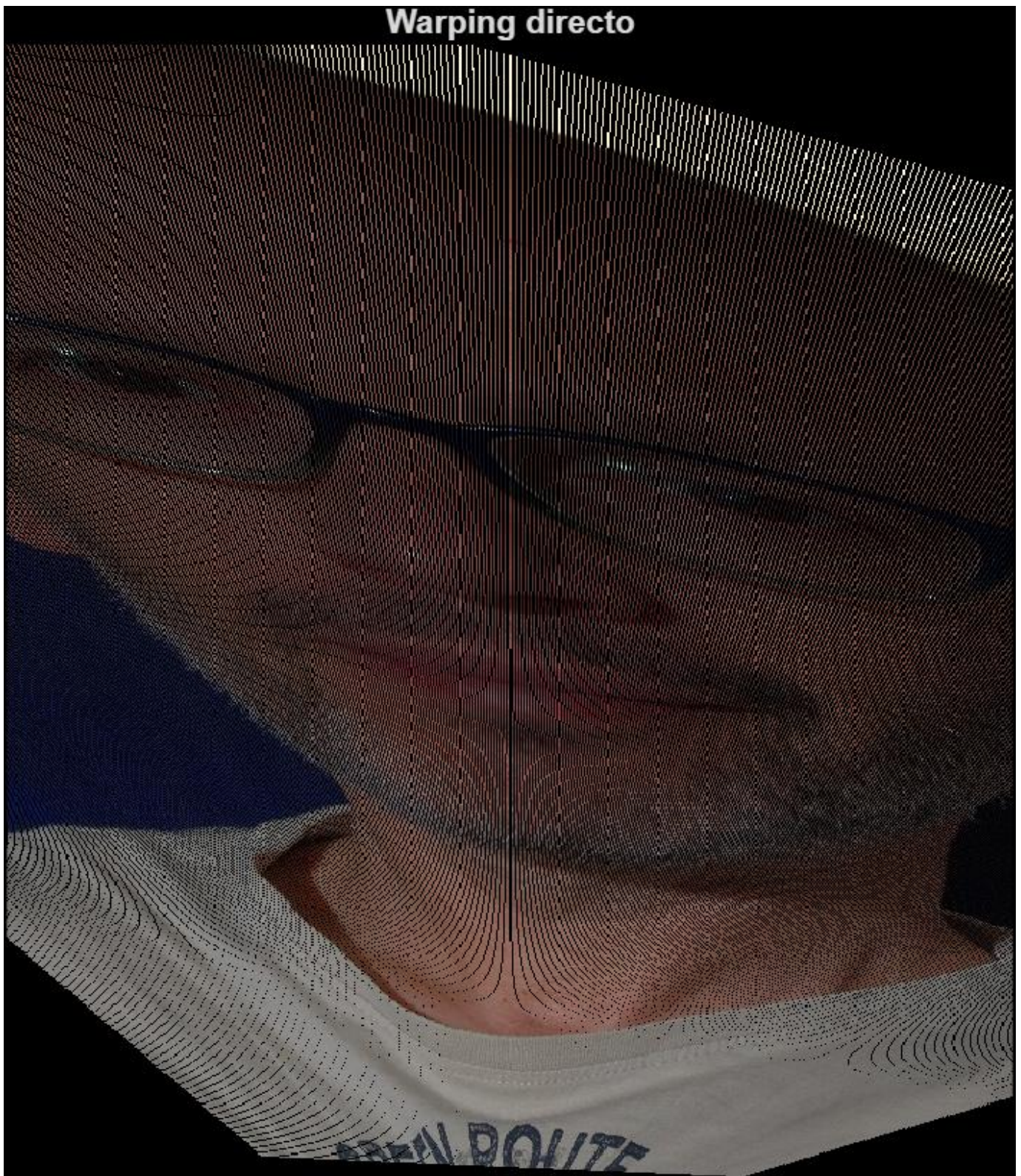
Punto regreso:

756.3684 171.4749

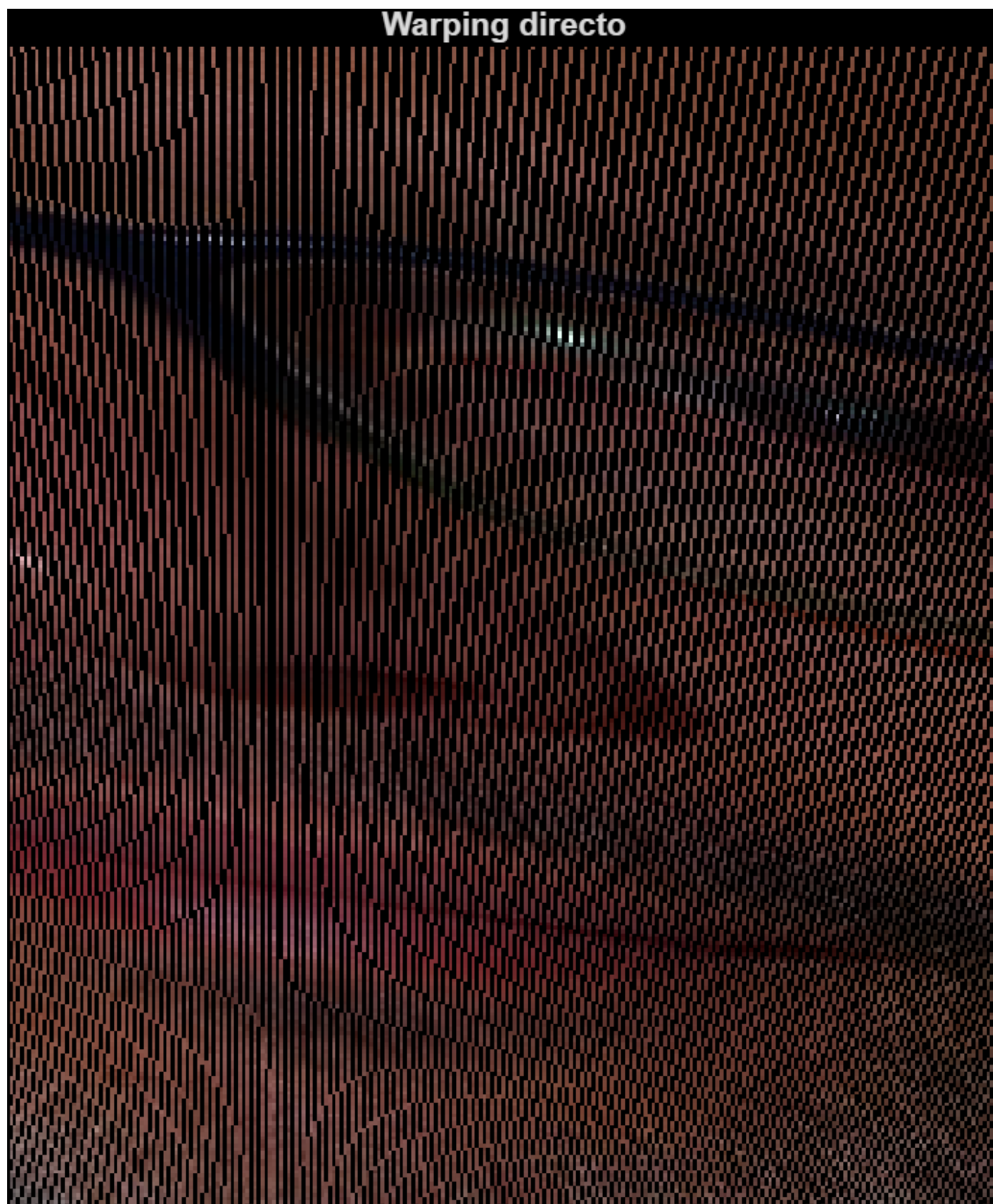
Adjuntad código de la función `warp_directo()` y una captura del resultado.

```
function im2 = warp_directo(im,P)
```

```
[N,M,C] = size(im);  
im2 = zeros(N,M,C);  
for y = 1:N  
    for x = 1:M  
        [u,v] = convertir(x,y,P);  
        u = round(u);  
        v = round(v);  
        if u >= 1 && u <= M && v >= 1 && v <= N  
            im2(v,u,:) = im(y,x,:);  
        end  
    end  
end  
end
```

Haced zoom sobre la imagen obtenida. ¿A qué se creéis que se deben los píxeles negros que aparecen en la imagen resultado?



Los pixeles negros aparecen por que hay zonas de la imagen destino que quedan sin un valor correspondiente despues de aplicar las tranformaciones.